



Gestion de fichiers

LARAVEL

Introduction

Laravel fournit une abstraction puissante du système de fichiers grâce au merveilleux package PHP **Flysystem** de Frank de Jonge. L'intégration de **Laravel Flysystem** fournit des pilotes simples pour travailler avec des systèmes de fichiers locaux, SFTP et Amazon S3. Mieux encore, il est incroyablement simple de passer d'une option de stockage à l'autre entre votre machine de développement locale et votre serveur de production, car l'API reste la même pour chaque système.

Configuration



Configuration

Le fichier de configuration du système de fichiers de Laravel se trouve à l'adresse **config/filesystems.php**. Dans ce fichier, vous pouvez configurer tous les "**disques**" de votre système de fichiers. Chaque disque représente un pilote de stockage particulier et un emplacement de stockage.

Le pilote local interagit avec les fichiers stockés localement sur le serveur exécutant l'application Laravel, tandis que le pilote s3 est utilisé pour écrire sur le service de stockage d'Amazon Cloud S3.

Le pilote local

Lorsque vous utilisez le pilote **local**, toutes les opérations sur les fichiers sont relatives au répertoire racine défini dans votre fichier de configuration des systèmes de fichiers. Par défaut, cette valeur est définie sur le répertoire **storage/app**.

Par conséquent, la méthode suivante écrira dans l'emplacement **storage/app/example.txt** :

```
use Illuminate\Support\Facades\Storage;  
  
Storage::disk('local')->put('example.txt', 'Contents');
```

Le disque public

Le disque **public** inclus dans le fichier de configuration des systèmes de fichiers de votre application est destiné aux fichiers qui seront accessibles au public. Par défaut, le disque public utilise le pilote local et stocke ses fichiers dans **storage/app/public**.

Pour rendre ces fichiers accessibles depuis le web, vous devez créer un lien symbolique de **public/storage** vers **storage/app/public** en utilisant la commande :

```
php artisan storage:link
```

Le disque public

Une fois qu'un fichier a été stocké et que le lien symbolique a été créé, vous pouvez créer une URL vers les fichiers à l'aide de l'assistant de ressources :

```
echo asset('storage/file.txt');
```

Vous pouvez configurer des liens symboliques supplémentaires dans votre fichier de configuration des systèmes de fichiers. Chacun des liens configurés sera créé lorsque vous exécuterez la commande **storage:link**.

```
'links' => [public_path('storage') => storage_path('app/public')]
```

Configuration du pilote FTP

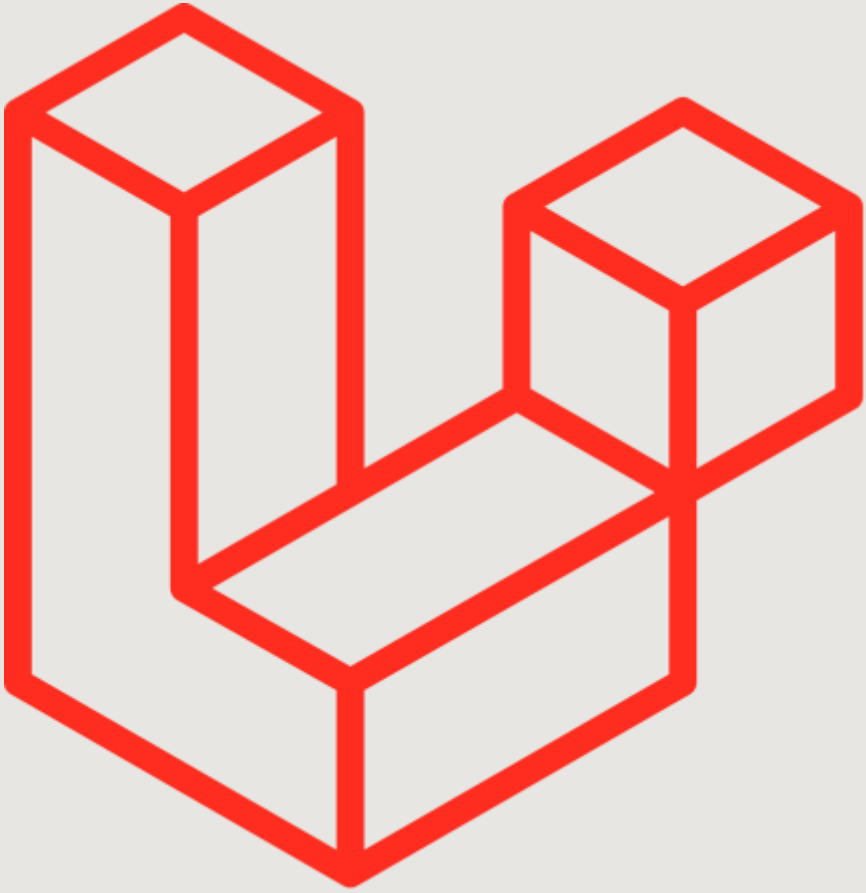
Avant d'utiliser le pilote FTP, vous devez installer le paquet FTP de Flysystem via le gestionnaire de paquets Composer :

```
composer require league/flysystem-ftp "^3.0"
```

vous pouvez utiliser l'exemple de configuration ci-dessous :

```
'ftp' => [  
    'driver' => 'ftp',  
    'host' => env('FTP_HOST'),  
    'username' => env('FTP_USERNAME'),  
    'password' => env('FTP_PASSWORD'),  
]
```

Obtention d'instances de disque



Obtention d'instances de disque

La façade **Storage** peut être utilisée pour interagir avec n'importe lequel de vos disques configurés. Par exemple, vous pouvez utiliser la méthode **put** de la façade pour stocker un avatar sur le disque par défaut.

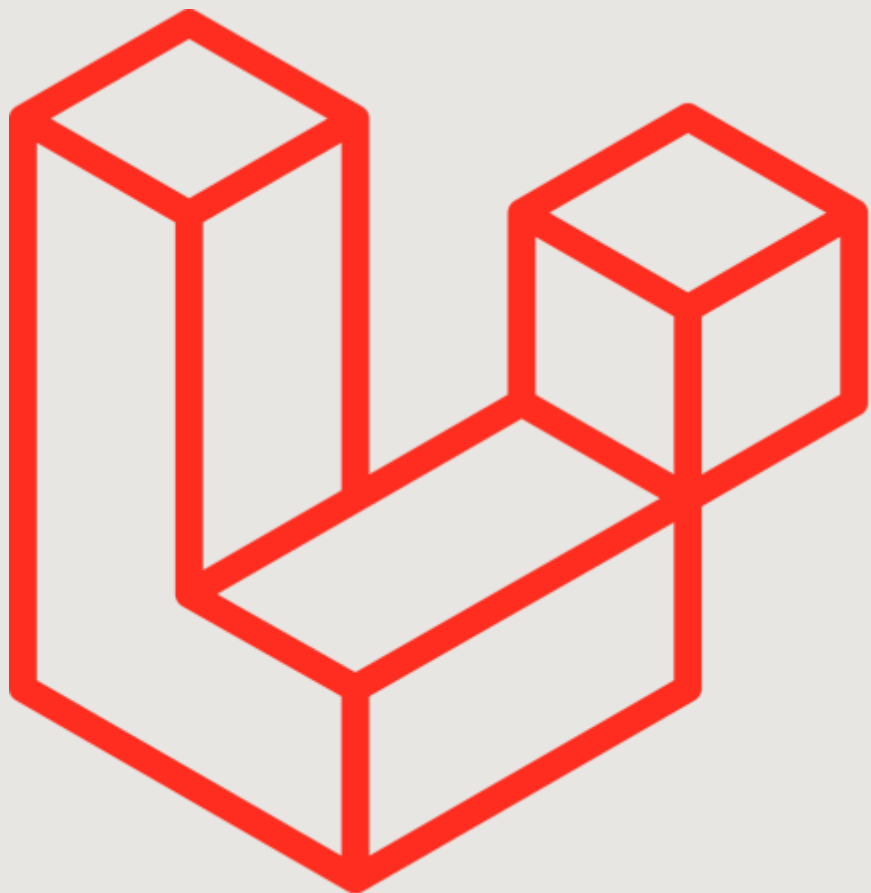
```
use Illuminate\Support\Facades\Storage;  
  
Storage::put('avatars/1', $content);
```

Obtention d'instances de disque

Si votre application interagit avec plusieurs disques, vous pouvez utiliser la méthode **disk** de la façade **Storage** pour travailler avec des fichiers sur un disque particulier :

```
use Illuminate\Support\Facades\Storage;  
  
Storage::disk('ftp')->put('avatars/1', $content);
```

Récupération des fichiers



Récupération des fichiers

La méthode **get** peut être utilisée pour récupérer le contenu d'un fichier. Le contenu brut du fichier sera renvoyé par la méthode. N'oubliez pas que tous les chemins d'accès aux fichiers doivent être spécifiés par rapport à l'emplacement "**racine**" du disque :

```
$contents = Storage::get('file.jpg');
```

Récupération des fichiers

La méthode **exists** peut être utilisée pour déterminer si un fichier existe sur le disque :

```
if (Storage::exists('file.jpg')) {  
    // ...  
}
```

La méthode **missing** peut être utilisée pour déterminer si un fichier est manquant sur le disque :

```
if (Storage::missing('file.jpg')) {  
    // ...  
}
```

Téléchargement de fichiers

La méthode **download** peut être utilisée pour générer une réponse qui oblige le navigateur de l'utilisateur à télécharger le fichier à l'emplacement indiqué. La méthode **download** accepte un nom de fichier comme deuxième argument , qui déterminera le nom de fichier qui sera vu par l'utilisateur téléchargeant le fichier. Enfin, vous pouvez transmettre un tableau d'en-têtes HTTP comme troisième argument de la méthode :

```
return Storage::download('file.jpg');  
return Storage::download('file.jpg', $name, $headers);
```

URL des fichiers

Vous pouvez utiliser la méthode **url** pour obtenir l'URL d'un fichier donné. Si vous utilisez le pilote **local**, il suffira généralement d'ajouter **/storage** au chemin d'accès donné et de renvoyer une URL relative vers le fichier. Si vous utilisez le pilote s3, l'URL distante entièrement qualifiée sera renvoyée :

```
use Illuminate\Support\Facades\Storage;
```

```
$url = Storage::url('file.jpg');
```


Métadonnées des fichiers

Laravel peut également fournir des informations sur les fichiers eux-mêmes. Par exemple, la méthode **size** peut être utilisée pour obtenir la taille d'un fichier en octets :

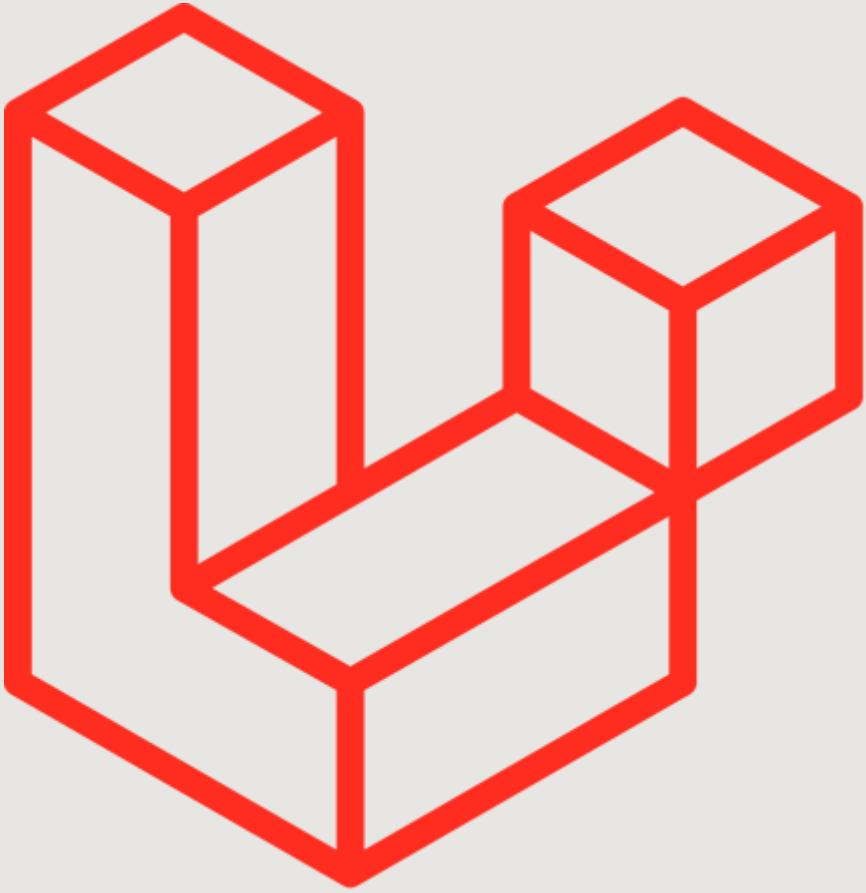
```
use Illuminate\Support\Facades\Storage;
```

```
$size = Storage::size('file.jpg');
```

```
$time = Storage::lastModified('file.jpg');
```

```
$path = Storage::path('file.jpg');
```

Stockage des fichiers



Stockage des fichiers

La méthode **put** peut être utilisée pour stocker le contenu d'un fichier sur un disque.

```
use Illuminate\Support\Facades\Storage;
```

```
Storage::put('file.jpg', $contents);
```

```
Storage::put('file.jpg', $resource); //PHP stream resource
```

Échec des écritures

Si la méthode **put** (ou d'autres opérations d'écriture) est incapable d'écrire le fichier sur le disque, **false** sera retourné :

```
if (!Storage::put('file.jpg', $contents)) {  
    // The file could not be written to disk...  
}
```

Prépendre et ajouter aux fichiers

Les méthodes **prepend** et **append** vous permettent d'écrire au début ou à la fin d'un fichier :

```
Storage::prepend('file.log', 'Prepended Text'); //Écrire au début
```

```
Storage::append('file.log', 'Appended Text'); //Écrire à la fin
```

Copie et déplacement de fichiers

La méthode **copy** peut être utilisée pour copier un fichier existant vers un nouvel emplacement sur le disque, tandis que la méthode **move** peut être utilisée pour renommer ou déplacer un fichier existant vers un nouvel emplacement :

```
Storage::copy('old/file.jpg', 'new/file.jpg');
```

```
Storage::move('old/file.jpg', 'new/file.jpg');
```

Téléchargements de fichiers (upload)

Laravel permet de stocker très facilement les fichiers téléchargés en utilisant la méthode **store** sur une instance de fichier téléchargé. Appelez la méthode **store** avec le chemin dans lequel vous souhaitez stocker le fichier téléchargé :

```
$path = $request->file('avatar')->store('avatars');
```

// ou bien

```
$path = Storage::putFile('avatars', $request->file('avatar'));
```

Spécifier un nom de fichier

Si vous ne souhaitez pas qu'un nom de fichier soit automatiquement attribué à votre fichier stocké, vous pouvez utiliser la méthode **storeAs**, qui reçoit le chemin d'accès, le nom de fichier et le disque (facultatif) comme arguments :

```
$path = $request->file('avatar')->storeAs(
    'avatars', $request->user()->id
);
```

```
$path = Storage::putFileAs(
    'avatars', $request->file('avatar'), $request->user()->id
);
```


Suppression de fichiers

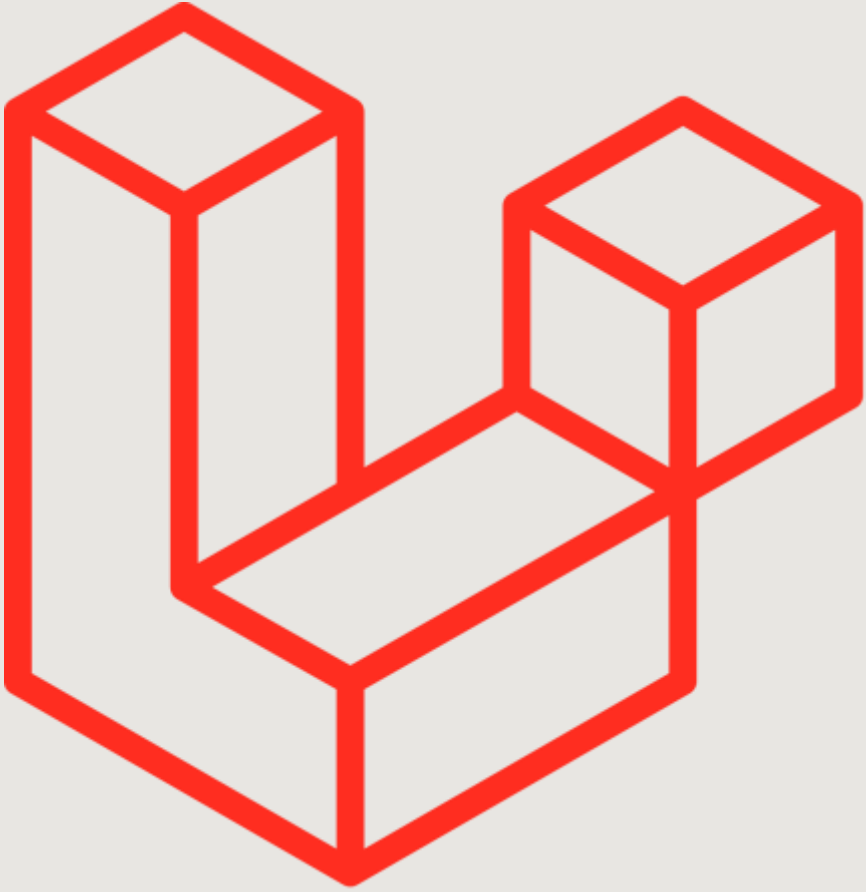
La méthode **delete** accepte un seul nom de fichier ou un tableau de fichiers à supprimer :

```
use Illuminate\Support\Facades\Storage;
```

```
Storage::delete('file.jpg');
```

```
Storage::delete(['file.jpg', 'file2.jpg']);
```

Les dossiers



Obtenir tous les fichiers d'un répertoire

La méthode **files** renvoie un tableau de tous les fichiers d'un répertoire donné. Si vous souhaitez récupérer une liste de tous les fichiers d'un répertoire donné, y compris tous les sous-répertoires, vous pouvez utiliser la méthode **allFiles** :

```
use Illuminate\Support\Facades\Storage;
```

```
$files = Storage::files($directory);
```

```
$files = Storage::allFiles($directory);
```

Obtenir tous les dossiers d'un dossier

La méthode **directories** renvoie un tableau de tous les répertoires d'un répertoire donné. En outre, vous pouvez utiliser la méthode **allDirectories** pour obtenir une liste de tous les répertoires d'un répertoire donné et de tous ses sous-répertoires :

```
use Illuminate\Support\Facades\Storage;
```

```
$directories = Storage::directories($directory);
```

```
$directories = Storage::allDirectories($directory);
```

Créer un répertoire

La méthode **makeDirectory** crée le répertoire donné, y compris tous les sous-répertoires nécessaires :

```
use Illuminate\Support\Facades\Storage;  
  
Storage::makeDirectory($directory);
```

Supprimer un répertoire

La méthode **deleteDirectory** peut être utilisée pour supprimer un répertoire et tous ses fichiers :

```
use Illuminate\Support\Facades\Storage;  
  
Storage::deleteDirectory($directory);
```